

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Нейроінформатика»

Робота №2

Тема «Нейромережі зі зворотними зв'язками»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Афанасьєва А.В.

2025

МЕТА

Вивчити моделі і методи навчання нейромереж зі зворотними зв'язками, розглянути приклади їхнього практичного використання; порівняти їхні можливості з можливостями нейромереж прямого поширення; ознайомитися зі стандартними програмними засобами для моделювання мереж зі зворотними зв'язками.

ЗАВДАННЯ

Завдання на роботу:

1. Ознайомитися з конспектом лекцій та рекомендованою літературою.
2. Вивчити архітектуру нейромереж Хопфілда та Ельмана, а також методи їхнього навчання.
3. Використовуючи документацію рекомендованого викладачем програмного засобу, вивчити його архітектуру і компоненти, призначені для моделювання і навчання нейромереж зі зворотними зв'язками, введення і виведення даних, підготовки звітів (виведення і відображення результатів роботи).
4. Визначити за таблицею 1.1 згідно з номером варіанта студента (обирається за журналом) номер підрозділу Додатку В, що містить опис прикладної задачі, та номери контрольних питань до даної роботи, відповіді на які належить навести у звіті.

Задача: В.2; Номер контрольних питань: 3, 13, 29, 33, 49

Б.2 Прогнозування надійності виробів електронної техніки

Вироби електронної техніки (ОСТ 11 ОДО.052.005), далі ВЕТ,

являють собою ненадійні вироби, що можуть бути охарактеризовані наступними параметрами:

- x1 – струм сітки першої зворотний, мкА;
- x2 – струм сітки першої зворотний при перегарті, мкА;
- x3 – струм анода, мА;
- x4 – струм сітки другий, мА;
- x5 – крутість характеристики, мА/В;
- x6 – крутість характеристики при недожару, мА/В;
- x7 – струм розжарення, мА;
- x8 – ємність вхідна, пф;
- x9 – ємність вихідна, пф;
- x10 – ємність прохідна, пф;
- x11 – час готовності, с;
- x12 – коефіцієнт підсилення в тріодному включенні;
- x13 – напруга контактної різниці потенціалів, В;
- x14 – відносна зміна струму анода, %;
- x15 – струм емісії при недожару, А;
- x16 – опір оксидного шару, Ом;
- x17 – струм емісії, А;
- x18 – відносна зміна крутості характеристики після іспитів на довговічність 5000 год., %.

Будучи піддані навантаженню в процесі експлуатації ці вироби можуть виходити з ладу в різний час. Задачею контролю якості цих виробів при завершенні виробництва, а також в експлуатації є прогнозування ресурсу (терміну служби) ВЕТ. Причому на практиці цілком достатньо якісного прогнозу, тобто досить спрогнозувати, чи зможе конкретний виріб прослужити заданий термін чи ні. Для побудови моделі якісної залежності між параметрами і класом ВЕТ можна використовувати експериментально отримані і пронормовані дані.

Формалізувавши задачу маємо: 18 вхідних ознак, що містять параметри ВЕТ 1 цільову ознаку - номер класу ВЕТ (1 - дефектний, 0 - гідний).

Кількість шарів: 2, кількість нейронів у шарах: 8-1, назва функції активації: логістична сигмоїдна.

5. Використовуючи Python для відповідних варіантів студента вхідних даних вирішити прикладну задачу на основі нейромереж Хопфілда та Ельмана.

6. Зберегти у файлі на диску початкові параметри нейромоделей, результати їхнього навчання (матрицю ваг та структуру з значенням функцій активації і кількості нейроелементів у шарах) та робити для навчальної та контрольної вибірок (помилку класифікації (оцінювання), значення на входах і виході, час навчання, час класифікації).

7. Результати виконання пунктів 5-6 занести в таблицю, стовпці якої повинні мати назви: назва архітектури нейромережі, кількість шарів, кількість нейронів у шарах, функції активації нейронів у шарах, метод навчання, час навчання, час класифікації навченої мережі для навчальної вибірки, помилка класифікації для навчальної вибірки, час класифікації навченої мережі для тестової вибірки, помилка класифікації для тестової вибірки.

8. Проаналізувати отримані результати і дати порівняльну характеристику мереж Хопфілда та Ельмана. Дати порівняльну характеристику мереж зі зворотними зв'язками та мереж прямого поширення за швидкістю навчання і точністю класифікації; принципами побудови архітектури.

9. Відповісти на контрольні питання.

10. Оформити звіт з роботи.

ВИКОНАННЯ

Код

Хопфілд

```
import numpy as np
import random

X = np.random.rand(500, 18)
y = [random.choice([-1, 1]) for _ in range(500)]

for row_i, row in enumerate(X):
    row_average = np.mean(row)

    for col_i, el in enumerate(row):
        if el < row_average:
            X[row_i][col_i] = -1
        elif el >= row_average:
            X[row_i][col_i] = 1

# Об'єднання (створення еталонів)
# по першим зразкам кожного класу
prototype_plus = None
prototype_minus = None

for i in range(len(X)):
    if y[i] == 1 and prototype_plus is None:
        prototype_plus = X[i].copy()
    elif y[i] == -1 and prototype_minus is None:
        prototype_minus = X[i].copy()
    if prototype_plus is not None and prototype_minus is not None:
        break

print("Еталон класу +1:", prototype_plus)
print("Еталон класу -1:", prototype_minus)

# --- Хопфілд ---
class HopfieldNetwork:
    def __init__(self, n_neurons):
        self.n = n_neurons
        self.W = np.zeros((n_neurons, n_neurons))

    def train(self, patterns):
        """Запам'ятовує еталони"""
        for p in patterns:
            p = p.reshape(-1, 1)
            self.W += p @ p.T
        np.fill_diagonal(self.W, 0) # немає зв'язків нейрона з собою
        self.W /= len(patterns)

    def predict(self, x, max_iters=50):
        """Відновлює найближчий еталон"""
```

```

        x = x.copy()
        for _ in range(max_iters):
            x_old = x.copy()
            for i in range(self.n):
                h = self.W[i] @ x
                x[i] = 1 if h >= 0 else -1
            if np.array_equal(x, x_old):
                break
        return x

# Навчання мережі
hopfield = HopfieldNetwork(n_neurons=18)
hopfield.train([prototype_plus, prototype_minus])

def classify(vector):
    restored = hopfield.predict(vector)

    # Відстань Хеммінга до кожного еталону
    dist_to_plus = np.sum(restored != prototype_plus)
    dist_to_minus = np.sum(restored != prototype_minus)

    return 1 if dist_to_plus < dist_to_minus else -1

print("\n" + "=" * 50)
print("ПЕРЕВІРКА ТОЧНОСТІ")
print("=" * 50)

correct = 0
for i in range(len(X)):
    pred = classify(X[i])
    if pred == y[i]:
        correct += 1

accuracy = correct / len(X) * 100
print(f"Точність на навчальних даних: {accuracy:.1f}%")

```

Ельман

```

from matplotlib import pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.optim as optim

INSTANCES = 500
EPOCHS = 500
print(f"{INSTANCES=}, {EPOCHS=}")

np.random.seed(42)

```

```

torch.manual_seed(42)

# Генерація даних
X = np.random.rand(INSTANCES, 18)
y = np.random.randint(0, 2, INSTANCES)

# Розділення вибірки на навчальну та тестову
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=0)

# Гіперпараметри
input_size = 1
hidden_size = 18
output_size = 1
epochs = EPOCHS
learning_rate = 0.01

# Дані для RNN потрібно перетворити у формат (samples, timesteps, features)
X_train_rnn = X_train.reshape(-1, 18, 1)
X_test_rnn = X_test.reshape(-1, 18, 1)
# 18 ознак → 18 часових кроків, 1 ознака на кроці

# Перетворення на тензори
X_train_tensor = torch.tensor(X_train_rnn, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
X_test_tensor = torch.tensor(X_test_rnn, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)

# Нейромережа Ельмана
class ElmanNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(ElmanNN, self).__init__()
        self.hidden_size = hidden_size
        self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
        self.h2o = nn.Linear(hidden_size, output_size)

        self.tanh = nn.Tanh()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x, hidden):
        # Об'єднання поточного входу і прихованого стану
        combined = torch.cat((x, hidden), dim=1)
        hidden = torch.tanh(self.i2h(combined))
        output = self.sigmoid(self.h2o(hidden))
        return output, hidden

# Ініціалізація мережі
model = ElmanNN(input_size, hidden_size, output_size)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# --- Навчання ---
train_losses = []

```

```

for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()

    total_loss = 0
    for i in range(len(X_train_tensor)):
        hidden = torch.zeros(1, hidden_size)

        # 18 ознак послідовно
        for t in range(18):
            current_input = X_train_tensor[i, t, :].view(1, -1) # (1, 1)
            output, hidden = model(current_input, hidden)

        # Втрати (тільки останній вихід)
        loss = criterion(output, y_train_tensor[i].view(1, -1))
        total_loss += loss

    # Зворотне поширення
    total_loss.backward()
    optimizer.step()

    avg_loss = total_loss.item() / len(X_train_tensor)
    train_losses.append(avg_loss)

# Перевірка результату
print("\nОцінка моделі:")
print("-" * 40)

model.eval()

# --- Навчальні дані ---
train_correct = 0
with torch.no_grad():
    for i in range(len(X_train_tensor)):
        hidden = torch.zeros(1, hidden_size)
        for t in range(18):
            current_input = X_train_tensor[i, t, :].view(1, -1)
            output, hidden = model(current_input, hidden)
        pred = 1 if output.item() >= 0.5 else 0
        if pred == y_train[i]:
            train_correct += 1

train_acc = train_correct / len(X_train_tensor)
print(f"Точність на навчальних даних: {train_acc * 100:.2f}%")

# --- Тестові дані ---
test_correct = 0
with torch.no_grad():
    for i in range(len(X_test_tensor)):
        hidden = torch.zeros(1, hidden_size)
        for t in range(18):
            current_input = X_test_tensor[i, t, :].view(1, -1)
            output, hidden = model(current_input, hidden)
        pred = 1 if output.item() >= 0.5 else 0
        if pred == y_test[i]:
            test_correct += 1

```



```
test_acc = test_correct / len(X_test_tensor)
print(f"Точність на тестових даних: {test_acc * 100:.2f}%")
```

Результати

Результати виконання роботи в консолі.

Хопфілд

```
Еталон класу +1: [-1.  1. -1. -1.  1. -1.  1. -1.  1.  1. -1. -1. -1.  1. -1.
1.  1.  1.]
Еталон класу -1: [ 1. -1. -1.  1. -1.  1.  1. -1. -1.  1. -1.  1.  1.  1.  1.
1.  1. -1.]
```

```
=====
ПЕРЕВІРКА ТОЧНОСТІ
=====
Точність на навчальних даних: 50.0%
```

Ельман

INSTANCES=500, EPOCHS=500

Оцінка моделі:

```
-----
Точність на навчальних даних: 95.71%
Точність на тестових даних: 52.00%
```

Таблиця Ельмана

Під час виконання роботи було сформовано таблицю порівняння параметрів мережі Ельмана, її наведено нижче:

Таблиця 1.1 — Параметри та результати нейромережі Ельмана

Кількість екземплярів	Кількість епох	Розподіл навчальної та тестової	Результати точності на	Результати точності на
-----------------------	----------------	---------------------------------	------------------------	------------------------

		вибірок (навчальна / тестова)	навчальний	тестовий
500	500	80/20	73.75%	48%
500	500	70/30	95.71%	52%
1 000	500	70/30	81.57%	55.33%
5 000	500	70/30	65.6%	50.07%
500	200	70/30	81.43%	52%